

Name :- N. Punith Kumar

Reg NO :- 1985A5156

course code :- CSA0307 - Data structure
& Name

COL4-AT1 - Comparative Analysis.

- 1) Single-threaded Quick sort vs
parallel Quick sort vs parallel
Merge sort

Amdahl's Law

The theoretical speedup for N threads
is:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}$$

where p is the fraction that can
be parallelized

for 8 threads, assuming 90% of work

$$S(8) = \frac{1}{0.1 + \frac{0.9}{8}} = \frac{1}{0.2125} \approx 4.71$$

so, the ideal maximum speedup
is about 4.71x, not 8x.

comparison :-

Factor

single-thread
Quick sort

Parallelism

None

Average comple
xity

$O(n \log n)$

Partition/merge
overhead

Low

Load balancing	Not applicable
synchronization	very Low
memory usage	Low
memory bandwidth	Low
scalability	poor

Parallel Quick sort

- The array is divided using a pivot.
- Different partitions can be processed simultaneously.
- problem: a poor pivot can create unequal partitions.
- This causes load imbalance, where some threads finish early while others continue.
- synchronization is required when partitions are created or combined.
- memory contention increases when 8 threads access the shared memory.

parallel merge sort

- The Input is divided into nearly equal portions.
- each thread sorts its portion independently.
- sorted portions are then merged.
- equal-sized partitions provide better load balancing.
- Its predictable divide-and-conquer structure makes parallelization easier.
- However, merging requires additional memory & creates memory-bandwidth pressure.

which scales better?

Parallel merge sort generally scales better than parallel quick sort on a shared-memory multicore system.

Reason:-

1. Better load balancing.
2. more predictable parallel decomposition.

3. Less dependence on pivot selection
4. more regular memory access.
5. easier synchronization b/w stages

conclusion:-

single-threaded quick sort has no-parallel speedup. parallel quick sort provides good speedup but suffers from pivot-related Load imbalance. parallel merge sort usually provides more consistent scaling with 8 threads. although memory bandwidth.

2) Map Reduce sorting vs parallel merge sort for 10 TB

Given:

- data = 10 TB
- Nodes = 100
- Average data per node:

$$\frac{10 \text{ TB}}{100} = 0.1 \text{ TB} = 100 \text{ GB}$$

so approximately 100 GB per node must be processed.

comparison :-

factor	mapreduce sorting
Architecture	Distributed
Data size	excellent for huge data.
Network communication	High
Disk I/O	very high
fault tolerance	Excellent.
scalability	excellent
synchronization	Distributed synchronization.
overhead	High
suitable for 10 TB/100 nodes	yes.

Map Reduce sorting

Map Reduce sorting generally works as follows:

Input $\rightarrow$ Map $\rightarrow$ Partition $\rightarrow$ shuffle $\rightarrow$ sort $\rightarrow$ Reduce $\rightarrow$ output

with total-order partitioning, the key space is divided into ordered ranges. each reducer receives a range of keys, so the final reducer outputs can be concatenated to form globally sorted data.

Advantages:

- data is distributed across 100 nodes.
- processing happens in parallel.
- faulty nodes can be handled by re-executing failed tasks.
- Disk-based processing allows datasets larger than RAM.
- Adding more nodes can increase capacity and throughput.

Disadvantages:-

- Shuffle phase generates significant network traffic.
- Intermediate data is written/read from disk.
- Startup and task scheduling overhead are high.

Parallel merge sort

on a shared-memory system, data can be divided among threads/nodes:

10TB $\rightarrow$ 100 partitions.

Final conclusion:-

for 10TB distributed across 100 nodes, MapReduce is usually the better choice when the priority is scalability, fault tolerance and handling data larger than a single machine's memory.